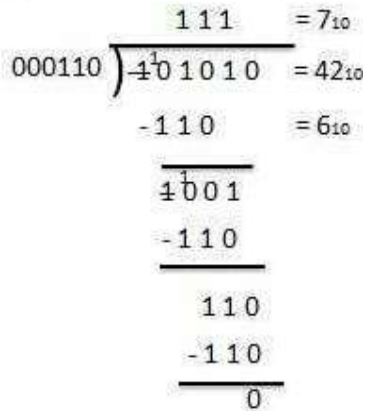
Binary Division

Binary division is similar to decimal division. It is called as the long division procedure.

Example Division

101010 / 000110 = 000111



Positive and Negative logic:

Binary logic is always 0s and 1s. There is only one little problem though, electricity doesn't come in 0s or 1s it comes in high or low voltage. So, you need to kind of assign which is which. And most of the time high voltage was assigned to be 1 and low voltage to be 0.

But you could also assign a high voltage to mean 0 and the low voltage to be 1 and there you have it: negative logic.

You can of course create whole circuits using negativ logic, but most of the time it's only used for things like enable inputs or similar. You can easily spot them by the line on top of the input name like: EN which would be an enable input that enables whatever can be enabled and disabled by that input with a low voltage on this input.

UNIT- 2

LOGIC GATES

Logic gates are the basic building blocks of any digital system. It is an electronic circuit having one or more than one input and only one output. The relationship between the input and the output is based on a **certain logic**. Based on this, logic gates are named as AND gate, OR gate, NOT gate etc.

AND Gate

A circuit which performs an AND operation is shown in figure. It has n input (n >= 2) and one output.

Y = A AND B AND C ..... N
Y = A.B.C ..... N
Y = ABC ..... N

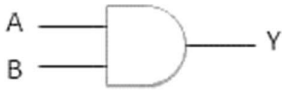


Figure 1.1: Logic diagram of AND gate

Table 1.13: Truth Table of AND gate

Inputs		Output
A	B	AB
0	0	0
0	1	0
1	0	0
1	1	1

OR Gate

A circuit which performs an OR operation is shown in figure. It has n input (n >= 2) and one output.

Y = A OR B OR C ..... N
Y = A + B + C ..... N



Figure 1.2: Logic diagram of OR gate

Table 1.14: Truth Table of OR gate

Inputs		Output
A	B	A + B
0	0	0
0	1	1
1	0	1
1	1	1

NOT Gate

NOT gate is also known as **Inverter**. It has one input A and one output Y.

Y = NOT A

Y = $\overline{A}$

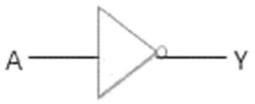


Figure 1.3: Logic diagram of NOT gate

Table 1.15: Truth Table of NOT gate

Inputs	Output
A	B
0	1
1	0

NAND Gate

A NOT-AND operation is known as NAND operation. It has n input (n >= 2) and one output.

Y = A NOT AND B NOT AND C ..... N

Y = A NAND B NAND C ..... N

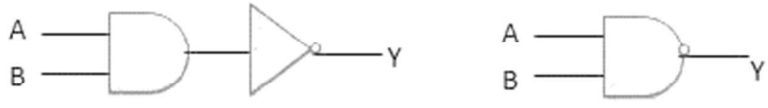


Figure 1.4: Logic diagram of NAND gate

Table 1.16: Truth Table of NAND gate

Inputs		Output
A	B	$\overline{AB}$
0	0	1
0	1	1
1	0	1
1	1	0

NOR Gate

A NOT-OR operation is known as NOR operation. It has n input (n >= 2) and one output.

Y = A NOT OR B NOT OR C ..... N
Y = A NOR B NOR C ..... N

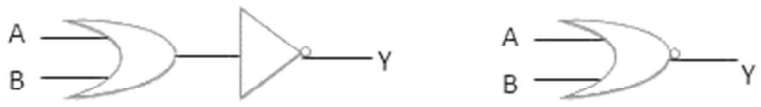


Figure 1.5: Logic diagram of NOR gate

Table 1.17: Truth Table of NOR gate

Inputs		Output
A	B	$\overline{A+B}$
0	0	1
0	1	0
1	0	0
1	1	0

XOR Gate

XOR or Ex-OR gate is a special type of gate. It can be used in the half adder, full adder and subtractor. The exclusive-OR gate is abbreviated as EX-OR gate or sometime as X-OR gate. It has n input (n >= 2) and one output.

$Y = A \text{ XOR } B \text{ XOR } C \ldots\ldots N$

$Y = A \oplus B \oplus C \ldots\ldots N$

$Y = \overline{A}B + A\overline{B}$



Figure 1.6: Logic diagram of XOR gate

Table 1.18: Truth Table of XOR gate

Inputs		Output
A	B	$A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

XNOR Gate

XNOR gate is a special type of gate. It can be used in the half adder, full adder and subtractor. The exclusive-NOR gate is abbreviated as EX-NOR gate or sometime as X-NOR gate. It has n input (n >= 2) and one output.

$Y = A \text{ XOR } B \text{ XOR } C \ldots\ldots N$

$Y = A \ominus B \ominus C \ldots\ldots N$

$Y = \overline{A}B + A\overline{B}$

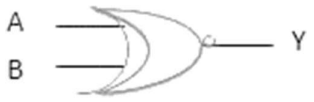


Figure 1.7: Logic diagram of XNOR gate

Table 1.19: Truth Table of XNOR gate

Inputs		Output
A	B	$A \ominus B$
0	0	1
0	1	0
1	0	0
1	1	1

BOOLEAN ALGEBRA

Boolean Algebra is used to analyze and simplify the digital (logic) circuits. It uses only the binary numbers i.e. 0 and 1. It is also called as Binary Algebra or logical Algebra. Boolean algebra was invented by George Boole in 1854.

Rule in Boolean Algebra

Following are the important rules used in Boolean algebra.

- Variable used can have only two values. Binary 1 for HIGH and Binary 0 for LOW.
- Complement of a variable is represented by an overbar (-). Thus, complement of variable B is represented as $\overline{B}$. Thus, if B = 0 then $\overline{B}$ = 1 and B = 1 then $\overline{B}$ = 0.
- ORing of the variables is represented by a plus (+) sign between them. For example, ORing of A, B, C is represented as A + B + C.
- Logical ANDing of the two or more variable is represented by writing a dot between them such as A.B.C. Sometime the dot may be omitted like ABC.

As well as the logic symbols “0” and “1” being used to represent a digital input or output, we can also use them as constants for a permanently “Open” or “Closed” circuit or contact respectively.

A set of rules or Laws of Boolean Algebra expressions have been invented to help reduce the number of logic gates needed to perform a particular logic operation resulting in a list of functions or theorems known commonly as the **Laws of Boolean Algebra**.

Boolean Algebra is the mathematics we use to analyse digital gates and circuits. We can use these “Laws of Boolean” to both reduce and simplify a complex Boolean expression in an attempt to reduce the number of logic gates required. *Boolean Algebra* is therefore a system of mathematics based on logic that has its own set of rules or laws which are used to define and reduce Boolean expressions.

The variables used in **Boolean Algebra** only have one of two possible values, a logic “0” and a logic “1” but an expression can have an infinite number of variables all labelled individually to represent inputs to the expression, for example, variables A, B, C etc, giving us a logical expression of A + B = C, but each variable can ONLY be a 0 or a 1.

Examples of these individual laws of Boolean, rules and theorems for Boolean Algebra are

given in the following table.

Table 1.20: Truth Tables for the Laws of Boolean

Boolean Expression	Description	Boolean Algebra Law or Rule
A + 1 = 1	A in parallel with closed = "CLOSED"	Annulment
A + 0 = A	A in parallel with open = "A"	Identity
A .1 = A	A in series with closed = "A"	Identity
A. 0 = 0	A in series with open = "OPEN"	Annulment
A + A = A	A in parallel with A = "A"	Idempotent
A . A = A	A in series with A = "A"	Idempotent
NOT $\overline{A}$ = A	NOT NOT A (double negative) = "A"	Double Negation
$\overline{A}$ + A = 1	A in parallel with NOT A = "CLOSED"	Complement
$\overline{A}$. A = 0	A in series with NOT A = "OPEN"	Complement
A+B = B+A	A in parallel with B = B in parallel with A	Commutative

$A.B = B.A$	A in series with B = B in series with A	Commutative
$\overline{A+B} = \overline{A}.\overline{B}$	invert and replace OR with AND	de Morgan's Theorem
$\overline{A.B} = \overline{A}+\overline{B}$	invert and replace AND with OR	de Morgan's Theorem

The basic **Laws of Boolean Algebra** that relate to the Commutative Law allowing a change in position for addition and multiplication, the Associative Law allowing the removal of brackets for addition and multiplication, as well as the Distributive Law allowing the factoring of an expression, are the same as in ordinary algebra.

Each of the Boolean Laws above are given with just a single or two variables, but the number of variables defined by a single law is not limited to this as there can be an infinite number of variables as inputs too the expression. These Boolean laws detailed above can be used to prove any given Boolean expression as well as for simplifying complicated digital circuits.

A brief description of the various **Laws of Boolean** are given below with A representing a variable input.

Description of the Laws of Boolean Algebra

Annulment Law – A term AND´ed with a "0" equals 0 or OR´ed with a "1" will equal 1

1. A . 0 = 0 A variable AND'ed with 0 is always equal to 0
2. A + 1 = 1 A variable OR'ed with 1 is always equal to 1

Identity Law – A term OR´ed with a "0" or AND´ed with a "1" will always equal that term

1. A + 0 = A A variable OR'ed with 0 is always equal to the variable
2. A . 1 = A A variable AND'ed with 1 is always equal to the variable

Idempotent Law – An input that is AND´ed or OR´ed with itself is equal to that input

1. A + A = A A variable OR'ed with itself is always equal to the variable
2. A . A = A A variable AND'ed with itself is always equal to the variable

Complement Law – A term AND´ed with its complement equals "0" and a term OR´ed with its complement equals "1"

1. $A . \bar{A} = 0$ A variable AND'ed with its complement is always equal to 0
2. $A + \bar{A} = 1$ A variable OR'ed with its complement is always equal to 1

Commutative Law – The order of application of two separate terms is not important.

1. A . B = B . A The order in which two variables are AND'ed makes no difference
2. A + B = B + A The order in which two variables are OR'ed makes no difference

Double Negation Law – A term that is inverted twice is equal to the original term

1. $\bar{\bar{A}} = A .$ A double complement of a variable is always equal to the variable

De Morgan´s Theorem – There are two "de Morgan´s" rules or theorems,

1. Two separate terms NOR´ed together is the same as the two terms inverted (Complement) and AND´ed for example: $\bar{\bar{A}} + \bar{\bar{B}} = \bar{A} . \bar{B}$
2. Two separate terms NAND´ed together is the same as the two terms inverted (Complement) and OR´ed for example: $\bar{\bar{A}} . \bar{\bar{B}} = \bar{A} + \bar{B}$

Distributive Law – This law permits the multiplying or factoring out of an expression.

1. A(B + C) = A.B + A.C (OR Distributive Law)
2. A + (B.C) = (A + B).(A + C) (AND Distributive Law)

Absorptive Law – This law enables a reduction in a complicated expression to a simpler one by absorbing like terms.

1. A + (A.B) = A (OR Absorption Law)
2. A(A + B) = A (AND Absorption Law)

Associative Law – This law allows the removal of brackets from an expression and regrouping of the variables.

1. A + (B + C) = (A + B) + C = A + B + C (OR Associate Law)
2. A(B.C) = (A.B)C = A . B . C (AND Associate Law)

Laws of Boolean Algebra

Using the above laws, simplify the following expression: (A + B)(A + C)

Q = (A + B).(A + C)

A.A + A.C + A.B + B.C – Distributive law

	A + A.C + A.B + B.C	– Idempotent AND law (A.A = A)
	A(1 + C) + A.B + B.C	– Distributive law
	A.1 + A.B + B.C	– Identity OR law (1 + C = 1)
	A(1 + B) + B.C	– Distributive law
	A.1 + B.C	– Identity OR law (1 + B = 1)
Q =	A + (B.C)	– Identity AND law (A.1 = A)

Then the expression: (A + B)(A + C) can be simplified to A + (B.C) as in the Distributive law.

POS and SOP Form Representations of a Boolean Function

There are three ways in which a Boolean function can be expressed which are as follows,

1. Product of Sum (POS) Form
2. Sum of Product (SOP) Form
3. Canonical Form of POS/SOP Form

1) Product of Sum (POS) Form

As the name suggests**,** A POS expression contains the sum of various terms ANDed/ multiplied together.

Example: Y = (A + B). (C + D). (E + F)

2) Sum of Product (SOP) Form

As the name suggests, A SOP expression is a group of product terms ORed/added together.

Example: Y = (A. B) + (C. D) + (E. F)

3) Canonical Form of POS/SOP Form

If each term of SOP/POS expression contains all the literals in the Boolean function, then they are said to be in canonical form. Suppose, we have a Boolean function (Y) having three literals A, B and C, then canonical Boolean expressions can be written as,

Y = ABC + A. B. C + A.B.C

This is an example of **canonical SOP Form**, because each term in the SOP form contains all

the literals **A**, **B** and **C**.

Similarly, Y = (A+B+C). (A + B + C). (A+B+C)

This is an example of **canonical POS form**, because each term of the POS form contains all the literals.

Now, let's see a few problems on canonical form.

Example 1: Convert the following expressions in their canonical form.

1. Y (A, B, C) = AB + BC + CA
2. Y (X, Y, Z) = X. (X + Y). (X + Y + Z)

Solution (i):

Y (A, B, C) = AB + BC + CA, this expression is a **SOP expression**, since we notice the Boolean function has three literals **A**, **B** and **C**, so each term of the Boolean expression must contain all the three literals to convert it into canonical SOP form. Therefore,

= Y (A, B, C) = AB + BC + CA

= AB. (C + C) + BC. (A + A) + CA. (B + B) [Since, C + C = 1]

= ABC + AB. C + ABC + A. BC + ABC + A. B. C

= ABC + AB. C + A. BC + A. B. C

Hence,

Y = ABC + AB. C + A. BC + A. B.C is the required canonical SOP form representation.

Solution (ii):

Y (X, Y, Z) = X. (X + Y). (X + Y + Z), is an example of **POS expression**, since all the sum terms in the expression doesn't have all the literals **X**, **Y** and **Z**, so we have to express it in such a way that it will have all the three literals in each term.

= Y (X, Y, Z) = X. (X + Y). (X + Y + Z)

= (X + Y. Y + Z. Z) (X + Y + Z. Z). (X + Y + Z)

= (X + Y. Y + Z) (X + Y. Y + Z) (X + Y + Z) (X + Y + Z) (X + Y + Z)

= (X + Y +Z) (X + Y + Z) (X + Y + Z) (X + Y + Z) (X + Y + Z) (X + Y + Z)

= (X + Y +Z) (X + Y + Z) (X + Y + Z) (X + Y + Z)

Hence,

Y = (X + Y +Z) (X + Y + Z) (X + Y + Z) (X + Y + Z) is the required canonical POS form representation.

KARNAUGH MAPS

Karnaugh maps provide a systematic method to obtain simplified sum-of-products (SOPs) Boolean expressions. This is a compact way of representing a truth table and is a technique that is used to simplify logic expressions. It is ideally suited for four or less variables, becoming cumbersome for five or more variables. Each square represents either a minterm or maxterm. A K-map of n variables will have 2